

Testing / Prueba

Proceso [PUD]

- Req's
- Análisis
- Diseño

- Implementación → Versión de producto

[Prueba]
- Despliegue

Artefactos:

Plan de prueba

Planificación →

Diseño

→ Casos de prueba

↓
Ejecución

→ Reporte de defectos

↓
Cierre

Para las pruebas:

(Un error que no se puede reproducir)
no es un error.

La planificación y el diseño (scr. del software de testing) es lo que se puede adelantar, para el otro scr. nuevo, lo una versión de producto para probar.

Una vez que tengas los requerimientos podemos adelantar la planificación y diseño de pruebas los casos de prueba tienen que quedar escritos correctamente, si no no vas a poder probar.

- Testing metodológico Vs Adhoc

- Testing metodológico sigue un proceso, diseña casos de prueba y usa esos casos de prueba.

- El Adhoc es una forma de probar sin planificación, sin diseño de casos de prueba, simplemente probar el programa a "fuerza bruta". No es parte de la Metod. Es no sistemático, no metodológico, sin proceso.

[Si no hay casos de prueba no tiene sentido probar, es plata desperdiciada.]

En las pruebas se valida y se verifica. Validar: Ver si haces lo que tiene que hacer. Verificar: Ver si lo que tiene que hacer lo hace bien.

A yo hago los casos de prueba en función del código y no de los req. voy a poder verificar, pero no validar. En cambio, si los casos de prueba son en función de los requerimientos puedo validar y verificar. Solo muchísimo más caro corregir problemas en la validación que en la verificación.

- Error Vs Defecto

~~Defecto es un error que se puede reproducir.~~

Algo es un defecto si se trata de una etapa anterior, viene de la implementación, errores de etapas anteriores. Son los que salen malos.

Los errores no se tratan, se identifican en una etapa y se corrigen en la misma. Se suelen corregir con pruebas automáticas. Son los buenos.

[El testing detecta defectos, los errores se corrigen antes.]

La mayoría de las veces se planifican solo la implementación, la cual dura por el 35% del esfuerzo, el testing se lleva un 30%-50% del esfuerzo.

Edad del defecto: tiempo que transcurre desde que se ^{introduce} el problema hasta que se detecta. Cuanto mayor sea el tiempo, más grande es el costo, este costo se incrementa exponencialmente.

[Si el defecto produce un malfuncionamiento en el SW se considera una Palla,
hay defectos que no producen Pallas en el sistema]

↗ Lo angustia testing

- Severidad del defecto (en función del impacto que causa para poder usar el sistema)

- ↳ Bloqueante - Invalidante (no anda el sistema)
- ↳ Grave
- ↳ Leve
- ↳ Cosmética (interfaz, interfaz, ortografía, Pormitos de campo)

~~Cada equipo deberá definir cual es cada punto de severidad.~~

- Prioridad (el cliente tiene que estar educado para asignar prioridades coherentes)

- ↳ Alta
- ↳ Media
- ↳ Baja

- Niveles de testing

- ↳ Pruebas de aceptación de usuario → se hace en despliegue
- ↳ Sistema → se hace en la prueba / testing si o si
- ↳ Integración o de interfaces → se hace en testing o implementación
- ↳ Unitaria → se hace en la implementación

Entornos de SW (Los entornos ideales, no se suele dar esto)

- ↳ Desarrollo → pruebas de desarrollo o de integración
- ↳ Prueba → pruebas de sistema
- ↳ Producción → se deberían hacer pruebas de aceptación de usuario para minimizar riesgos
- ↳ Producción

Ciclo de prueba

El primer ciclo de prueba es ciclo 0, mandan una nueva versión de comp. ejecutan los casos de prueba para esta nueva versión y luego el ciclo 1. Así sucesivamente. Idealmente deberías tener dos ciclos de prueba, pero esto no suele ^{suceder} pasar.

Hay empresas que tienen muchos ciclos de testing para salir a producción y eso es común

Estrategias de prueba. (Una vez que paso el ciclo 0, me vuelvo a mandar una nueva versión y luego 2 estrategias)

↳ Sin Regresión (Punto los errores identificados en el reporte del ciclo 0 y me lo mandan)

↳ Con Regresión (Ejecutar de nuevo los casos de prueba para el ciclo 0 sin tener en cuenta el reporte de errores, de esa forma no se me pasan nuevos errores que se pudieron generar)

[Lo que problemas en testing son los requerimientos Funcionales y no Funcionales]

Tipos de prueba

- ↳ Pruebas de regresión en operación normal (Caso normal de uso) / Operación negativa (Caso de fallo)
- ↳ Pruebas de proceso de negocio (para requerimientos)
- ↳ Pruebas de la documentación (Manual de usuario y manual de configuración) (para req. Funcionales)
- ↳ Pruebas de performance
- ↳ Pruebas de interfaces
- ↳ Pruebas de casos completos → Probar lo máximo que se ve a poder pasar el límite del sistema sin que se rompa
- ↳ Pruebas de estrés → Se lleva al límite del sistema, no se espera que se cumpla la prueba. Es para ver cómo queda
- ↳ Pruebas de seguridad
- ↳ " " de accesibilidad → No solamente para personas con capacidades limitadas, si no en general. SW accesible para el ambiente que se está desarrollando.

Preg
No

Funcion

Es para ver cómo queda

Un caso de prueba es una situación descrita, detallada que apunta a hacer
verificación y validación de una determinada funcionalidad. Contiene un
escenario (paso a paso), ^(precondiciones, condiciones) datos de prueba y el resultado esperado.

Yo dejo de probar cuando (hay distintas estrategias para esto) podemos usar
matrices de porcentaje para cada tipo de error (0% bloqueante y así)

Cuando dejar de probar es un problema, y esto cambia para dejar de probar
se define en el plan de prueba. La estrategia a tomar se define en el plan de prueba.

Testing no está en condiciones de asegurar la calidad del producto o de que no haya
más errores. Testing garantiza solamente la falta de calidad